SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

Course Name:	Software Testing	**Course Code:**	CSA37
Student Name:	Ganesh Karthick M	**Reg. No.:**	192321138
Section / Batch:		**Date:**	24-10-2026

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

QUESTIONS: 15

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5
4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5

Sl. No	Questions	Bloom's Level
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5
9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5
15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Code of Conduct:

I Ganesh Karthick M / 192321138 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Annexure A – Scenario-Based Assessment: Answers

Where a question gives no numbers or code, the answer states its assumptions (limits, rules, snippets) and works the technique through on them.

Question 1: Online Banking Fund Transfers

Assumptions: a transfer is valid from 100 to 50,000 per transaction; the balance must cover the amount; the beneficiary account must be valid.

(a) Evaluation of the current strategy

Failures at the minimum and maximum limits show that the team tested typical amounts and some random amounts, but never the values *at* the limits, where off-by-one defects (< instead of ≤) occur. Random testing has no coverage measure and cannot be repeated or traced to requirements.

(b) Equivalence partitioning and boundary value analysis

Field	Partition	Representative	Valid?
Amount	Below minimum (< 100)	50	No
Amount	Within limits (100 to 50,000)	5,000	Yes
Amount	Above maximum (> 50,000)	60,000	No
Amount	Non-numeric / negative / empty	abc, −100	No
Balance	Balance ≥ amount	10,000	Yes
Balance	Balance < amount	4,000 for 5,000	No
Beneficiary	Valid / invalid account	valid / 00000	Yes / No

TC	Amount	Balance	Technique	Expected
1	99	10000	BVA min−	Rejected: below minimum
2	100	10000	BVA min	Accepted
3	101	10000	BVA min+	Accepted
4	5000	10000	ECP valid	Accepted
5	49999	60000	BVA max−	Accepted
6	50000	60000	BVA max	Accepted
7	50001	60000	BVA max+	Rejected: above maximum
8	5000	5000	Balance boundary	Accepted, balance 0
9	5000	4999	Balance − 1	Rejected: insufficient funds
10	−100 / abc	10000	Invalid class	Rejected: invalid amount

(c) Random testing: retain or replace?

If amounts are drawn uniformly from 1 to 1,00,000, the chance that one random test hits any of the 6 boundary values is 6 / 100,000 = 0.006%, so about 16,000 random tests would be needed on average to hit just one of them. **Decision:** replace random testing as the primary technique; keep a small random/exploratory session *after* the systematic tests, only to look for unexpected defects.

(d) Optimized approach

Condition	C1 amount in limits	C2 balance ≥ amount	C3 beneficiary valid	Action
R1	Y	Y	Y	Transfer succeeds
R2	Y	Y	N	Reject: invalid beneficiary
R3	Y	N	Y	Reject: insufficient funds
R4	Y	N	N	Reject (validation order defined)
R5	N	—	—	Reject: amount out of limits (4 rules collapse to 1)

Justification: 10 BVA/ECP cases plus 5 decision rules (about 15 tests) cover every partition, every boundary and every combination, which random testing cannot guarantee at any size. **Interpretation:** boundary defects are the most likely at limits, so BVA must be the core of the suite.

Question 2: Healthcare Module – Coverage Gaps

Assumed logic: validate patient data; for each prescribed medicine: if the patient is allergic raise an alert and skip it; otherwise calculate the dose and cap it at the safe maximum; finally save the prescription.

(a) Conceptual control flow graph

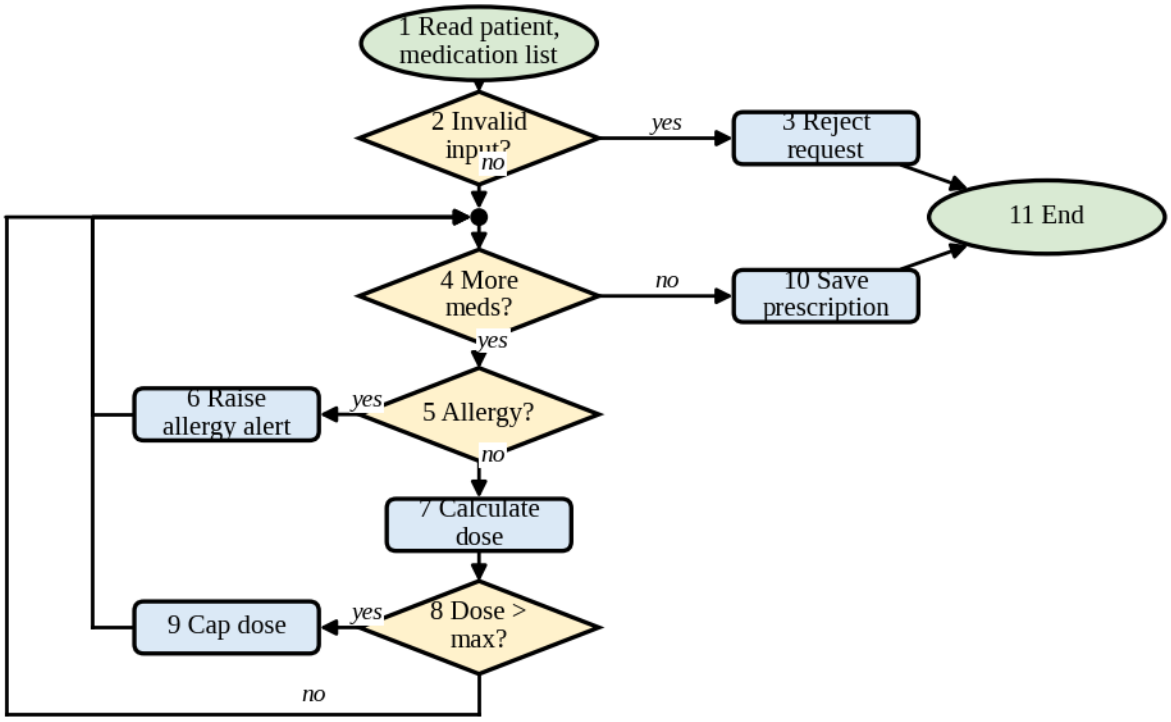


Figure 1: Control flow graph of the prescription module

E = 14, N = 11, V(G) = 14 − 11 + 2 = 5 (4 decisions + 1). Independent paths:

Path	Node sequence	Scenario
P1	1-2-3-11	Invalid input rejected
P2	1-2-4-10-11	No medicines, saved
P3	1-2-4-5-6-4-10-11	Allergy alert, medicine skipped
P4	1-2-4-5-7-8-4-10-11	Dose within maximum
P5	1-2-4-5-7-8-9-4-10-11	Dose capped at maximum

(b) Assessment of the coverage achieved

Assume the team ran two tests: a valid adult with no allergy and a normal dose (P4), and an invalid input (P1).

Criterion	Covered	Percentage	Missing
Statement (11 nodes)	9	82%	Nodes 6 (alert) and 9 (cap)
Branch (14 edges)	10	71%	5→6, 6→4, 8→9, 9→4
Independent paths (5)	2	40%	P2, P3, P5

The untested paths are the **allergy alert** and the **maximum-dose cap**, exactly the safety-critical logic where post-deployment defects would be most harmful.

(c) Improved strategy

TC	Input	Path	Expected
1	age = −1	P1	Rejected
2	valid patient, no medicines	P2	Saved with empty list
3	1 medicine, patient allergic	P3	Alert, medicine excluded
4	1 medicine, normal dose	P4	Dose as calculated
5	1 medicine, calculated dose > max	P5	Dose = max
6	3 medicines: normal, allergic, overdose	P4+P3+P5 in one loop	Each handled correctly
7	dose = max exactly; dose = max + 1	Boundary of decision 8	Not capped; capped
8	0, 1, 2 and many medicines	Loop boundary	All handled

Reliability and risk: the 8 tests give 100% statement, 100% branch and all 5 independent paths plus loop boundaries, so every allergy and overdose decision outcome has been exercised before release. **Interpretation:** coverage must be measured at branch/path level, not statement level, for safety-critical code.

Question 3: Retail E-commerce: Price, Quantity and Discount

Assumptions: price 1 to 10,000; quantity 1 to 100; discount 0 to 50%; a discount above 20% is allowed only for quantity ≥ 10 (bulk rule); total = price × quantity × (1 − discount).

(a) Limitations of the current approach

Each field was probably tested on its own with typical values. The defects are in **interactions** (here the bulk rule), which single-field tests cannot reveal.

(b) ECP and BVA per field

Field	Valid class	Invalid classes	BVA values (robust)
Price	1 to 10,000	< 1, > 10,000, non-numeric	0, 1, 2, 5,000, 9,999, 10,000, 10,001
Quantity	1 to 100	< 1, > 100, fractional	0, 1, 2, 50, 99, 100, 101
Discount %	0 to 50	< 0, > 50	−1, 0, 1, 25, 49, 50, 51

Full combination of the 3 classes per field gives 3 × 3 × 3 = 27 cases; all robust BVA combinations would be 7^3 = 343. Using the single-fault assumption, robust BVA needs only 6n + 1 = 19 cases for n = 3 fields.

(c) Decision table for the interaction

Rule	All fields valid	Discount > 20%	Quantity ≥ 10	Action
R1	N	—	—	Reject: invalid input
R2	Y	N	—	Accept; total = P × Q × (1 − D)
R3	Y	Y	Y	Accept (bulk discount)
R4	Y	Y	N	Reject: bulk quantity required

TC	Price	Qty	Discount	Rule	Expected
1	500	1	0%	R2	500
2	500	9	20%	R2 boundary	3,600
3	500	9	30%	R4	Rejected: bulk required
4	500	10	30%	R3	3,500
5	500	10	50%	R3 max	2,500
6	500	10	51%	R1	Rejected
7	10001	10	10%	R1	Rejected
8	500	0	10%	R1	Rejected

(d) Revised strategy and justification

Strategy: 19 single-fault robust BVA tests (fields at the boundary, others nominal) + 4 decision-table tests for the interaction + 3 pairwise combinations of boundary values (for example quantity 10 with discount 20/21%) = about 26 tests instead of 343. **Defect detection:** boundary defects are found by BVA, combination defects by the decision table (the case-3 rejection would have been missed by every isolated test). **Completeness:** all partitions, all boundaries and every rule of the business logic are covered. **Interpretation:** combine BVA/ECP for single-field limits with a decision table for field interactions.

Question 4: Nested Conditions and Loops – Partial Path Coverage

Assumed module (order processing):

```
1: i = 0
2: while (i < n) {
3:   if (order[i].valid) {
4:     if (order[i].express) ship = FAST; else ship = STD;
7:     if (order[i].total > 1000) applyBonus()
9:   } else logError()
10:   i++ }
11: return
```

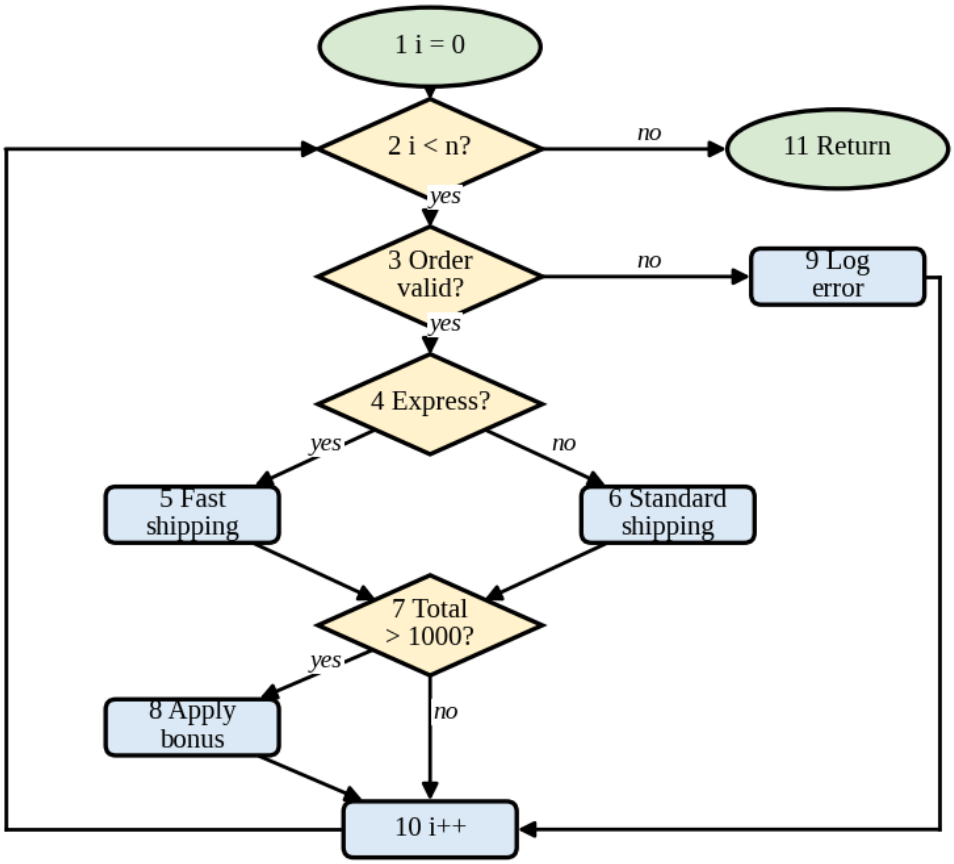


Figure 2: Control flow graph of the order-processing module

(a) Gaps in the current approach

Suppose the team's tests execute the standard and express shipping branches with small orders, which gives roughly 3 of the 5 independent paths (60% partial coverage). The graph shows the gaps: **n = 0** (path P1), **an invalid record** (P2) and the **bonus** branch (P5) were not identified when the CFG was built.

(b) Independent paths and cyclomatic complexity

E = 14, N = 11, V(G) = 14 − 11 + 2 = 5 (4 decisions + 1).

Path	Node sequence	Scenario	Test input	Expected
P1	1-2-11	n = 0	empty list	Returns, nothing processed
P2	1-2-3-9-10-2-11	Invalid order	1 invalid order	Error logged

Path	Node sequence	Scenario	Test input	Expected
P3	1-2-3-4-5-7-10-2-11	Express, total ≤ 1000	express, 800	FAST, no bonus
P4	1-2-3-4-6-7-10-2-11	Standard, total ≤ 1000	standard, 800	STD, no bonus
P5	1-2-3-4-5-7-8-10-2-11	Express, total > 1000	express, 1500	FAST, bonus applied

(c) Impact of missed paths

Each missed independent path hides at least one decision outcome: skipping P1 and P2 leaves empty-input and error-handling defects, skipping P5 leaves the bonus calculation unverified. With 5 independent paths and only 3 tested, up to 40% of the module's decision logic is unverified.

(d) Complete path testing strategy

Test	Input	Purpose	Expected
1 to 5	One order per path P1 to P5 as above	Basis paths	As in table
6	total = 1000 and 1001, express	Boundary of decision 7	No bonus; bonus
7	n = 1, 2 orders mixed (invalid, express, standard)	Loop 1 and 2 iterations, path combinations	All three handled in order
8	n = 1000 orders	Loop upper boundary	All processed, i ends at n

Coverage criteria and outcomes: tests 1 to 5 give 100% statement, 100% branch and all independent paths; tests 6 to 8 add the data boundary and loop boundary. **Interpretation:** counting independent paths from the CFG (V(G) = 5) tells the team exactly how many tests are needed and which are missing.

Question 5: Financial Application – Hybrid Black Box and White Box

(a) Critical analysis of the current integration

Weakness	Effect
Black-box and white-box suites designed separately	Overlapping tests and uncovered gaps; no traceability from requirement to code path
Statement coverage reported as the main metric	Hides untested branches and conditions
No boundary tests derived from requirements	Off-by-one defects reach production
White-box tests not used to find missing BVA cases and vice versa	Neither technique checks the other's blind spots

(b) Worked hybrid example: loan eligibility

Rules: age 21 to 60, income ≥ 25,000, credit score ≥ 700. Code: `if (age < 21 || age > 60) reject; else if (income >= 25000 && score >= 700) approve; else reject`. Decisions D1 (two conditions) and D2 (two conditions): V(G) = 2 + 1 = 3.

TC	Age	Income	Score	Black-box basis	White-box coverage	Expected
1	20	30000	750	BVA age min−	D1 true via age < 21	Reject
2	21	30000	750	BVA age min	D1 false	Approve
3	60	30000	750	BVA age max	D1 false	Approve
4	61	30000	750	BVA age max+	D1 true via age > 60	Reject
5	35	25000	700	BVA income and score min	D2 true	Approve
6	35	24999	750	BVA income min−	D2 false (income)	Reject
7	35	30000	699	BVA score min−	D2 false (score)	Reject

These 7 tests give all BVA boundaries, 100% branch coverage, all 3 independent paths (reject by age, approve, reject by D2) and MC/DC for both decisions (each condition changes the outcome independently: tests 1/2 and 4/3 for age, 5/6 and 5/7 for income and score).

(c) Improved hybrid strategy

Step	Activity	Adequacy criterion
1	Derive ECP and BVA cases from the requirements	100% of partitions and boundaries
2	Build the CFG and run the black-box cases with coverage tooling	Measure branch and condition coverage
3	Add white-box cases for uncovered branches, conditions and paths	100% branch, MC/DC on compound decisions
4	Review unreachable code and requirement gaps found in step 3	Zero unexplained gaps
5	Regression run and defect-trend review each release	No uncovered change

Justification: black-box tests guarantee that the *requirements* are tested; white-box measurement proves that the *code* is exercised and exposes what the requirements-based tests missed. Together they prevent the defects that either one alone leaves behind. **Conclusion:** use ECP and BVA to design, CFG-based measurement to verify, and add path/condition tests to close the gaps.

Question 6: HR Payroll Bonus: Exactly 5, 10 and 15 Years

Assumed bonus rule: 0 to 4 years: 5%; 5 to 9: 10%; 10 to 14: 15%; 15 to 45: 20% of monthly salary; payout only if rating is 3, 4 or 5; years valid 0 to 45; rating 1 to 5.

(a) Analysis of the failure

The failures occur only at 5, 10 and 15, the starting value of each tier, so the code most likely uses > instead of ≥ at the tier limits. For a salary of 50,000 the loss is:

Years	Expected bonus	Actual (defect)	Shortfall
5	10% = 5,000	5% = 2,500	2,500
10	15% = 7,500	10% = 5,000	2,500
15	20% = 10,000	15% = 7,500	2,500

The existing tests clearly used mid-tier values (for example 3, 7, 12, 20 years), which pass in both correct and faulty code.

(b) ECP and BVA redesign

Input	Partitions
Years	Invalid < 0 | 0 to 4 | 5 to 9 | 10 to 14 | 15 to 45 | Invalid > 45
Rating	Invalid < 1 | 1 to 2 (no bonus) | 3 to 5 (bonus) | Invalid > 5

Years	Tier tested	Expected %	Years	Tier tested	Expected %
−1	invalid	Rejected	11	10 to 14	15%
0	0 to 4 (min)	5%	14	10 to 14 (max)	15%
1	0 to 4	5%	15	15 to 45 (min)	20%
4	0 to 4 (max)	5%	16	15 to 45	20%
5	5 to 9 (min)	10%	45	15 to 45 (max)	20%
6	5 to 9	10%	46	invalid	Rejected
9	5 to 9 (max)	10%	10	10 to 14 (min)	15%

Rating boundaries: 0 (invalid), 1, 2 (no bonus), 3 (bonus), 5, 6 (invalid). **Combination:** years {4, 5, 9, 10, 14, 15} × rating {2, 3} = 12 tests to check that the tier boundary and the rating boundary do not interfere.

(c) Why BVA/ECP are superior to exploratory testing here

Aspect	Exploratory	BVA + ECP
Chance of finding the 3 buggy values	Picking 5 of 46 valid years at random finds at least one with probability 1 − C(43,5)/C(46,5) = 29.8%	100%: boundaries 5, 10, 15 are always tested
Repeatability	Low	High; same tests every release
Evidence of coverage	None	Every partition and boundary listed
Suitability for financial logic	Weak: errors are exact thresholds	Strong: targets thresholds directly

Interpretation: exploratory testing is useful for unexpected problems, but payroll thresholds demand systematic boundary tests because a single off-by-one error is paid out repeatedly to many employees.

Question 7: Laboratory Temperature Control – MC/DC

Assumed logic (Tmin = 20, Tmax = 60, Tcrit = 80 °C):

```
if (T > Tcrit || (s1Fault && s2Fault)) alarm = ON
else if (T < Tmin) heater = ON
else if (T > Tmax) cooler = ON
else idle
```

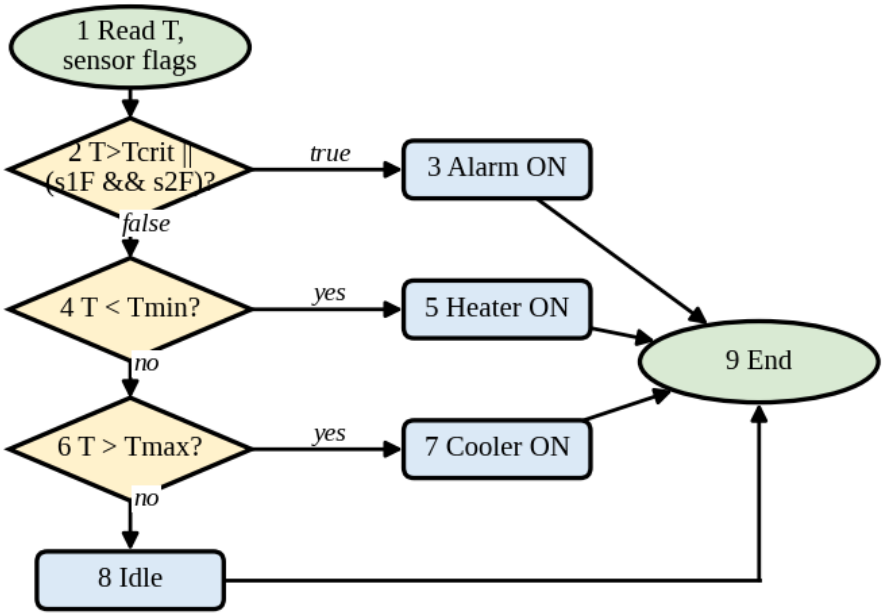


Figure 3: CFG of the temperature-control logic

E = 11, N = 9, V(G) = 11 − 9 + 2 = 4 (3 decisions + 1). Paths: P1 1-2-3-9 (alarm); P2 1-2-4-5-9 (heater); P3 1-2-4-6-7-9 (cooler); P4 1-2-4-6-8-9 (idle).

(a) Gap in branch coverage

Four tests (T = 90 with sensors OK, T = 10, T = 70, T = 40) take every branch outcome (decision 2 true and false, heater, cooler, idle) = 100% branch coverage. Yet decision 2 is true only because of T > Tcrit; **the sub-condition (s1Fault && s2Fault) was never evaluated as true**. A defect in that expression (for example using || or ignoring s2Fault) is invisible, which is exactly the sensor-malfunction failure seen after release.

(b) MC/DC strategy

Let A = T > Tcrit, B = s1Fault, C = s2Fault; D = A || (B && C). MC/DC needs n + 1 = 4 tests for 3 conditions:

Test	A	B	C	D (alarm)	Shows independence of
TC1	F	T	T	T	B and C (with TC3, TC2)
TC2	F	F	T	F	B (vs TC1)
TC3	F	T	F	F	C (vs TC1)
TC4	T	F	T	T	A (vs TC2)

Pairs: A: TC2/TC4 (B, C fixed, only A changes D); B: TC1/TC2; C: TC1/TC3. Add TC5 (T = 10, heater), TC6 (T = 70, cooler) and TC7 (T = 40, sensors OK, idle) so the remaining branches are covered: 7 tests give 100% MC/DC, 100% branch and all 4 independent paths.

(c) Impact on safety-critical reliability

MC/DC proves that every condition of the alarm decision can independently trigger the alarm, so a sensor-fault combination cannot silently be ignored. It needs only n + 1 tests per decision instead of the $2^n = 8$ of exhaustive testing, which keeps the effort practical. This is why standards for airborne software (DO-178C, level A) require MC/DC. **Interpretation:** for safety logic, branch coverage is not enough; MC/DC would have exposed the missed alarm before release.

Question 8: Travel Portal – Promo Code Combinations

Assumptions: promo A = 30% off; promo B = flat 1,000 off; Gold tier gives an extra 10% off (percentages add); total = price × (1 − percent) − flat. The total must never be negative (the minimum is 0).

(a) Limitation of testing inputs in isolation

Promo A alone, promo B alone and Gold alone each pass. The defect appears only when the flat discount is applied to a **low price after percentage discounts**: for price 900 with promo B alone the total is −100. Isolated tests never combine price level, promo type and tier.

(b) Decision table (conditions: A, B, Gold, price after % ≥ 1000)

4 binary conditions give 16 combinations. When B is not applied the price condition is irrelevant, so rules collapse to 4 + 8 = **12 rules**.

Rule	A	B	Gold	Price	Total formula	Expected
R1	N	N	N	5000	P	5,000
R2	N	N	Y	5000	0.9 P	4,500
R3	Y	N	N	5000	0.7 P	3,500
R4	Y	N	Y	5000	0.6 P	3,000
R5	N	Y	N	5000	P − 1000	4,000
R6	N	Y	Y	5000	0.9 P − 1000	3,500
R7	Y	Y	N	5000	0.7 P − 1000	2,500
R8	Y	Y	Y	5000	0.6 P − 1000	2,000
R9	N	Y	N	900	900 − 1000 = −100	0 (capped)
R10	N	Y	Y	900	810 − 1000 = −190	0 (capped)
R11	Y	Y	N	900	630 − 1000 = −370	0 (capped)
R12	Y	Y	Y	900	540 − 1000 = −460	0 (capped)

Rules R9 to R12 are the defect-revealing combinations: the current system would show negative totals.

(c) Optimized test set

Reduction step	Count
Exhaustive condition combinations	16
After collapsing don't-care price when B is not applied	12
Add boundary: price 1000 with B only (total 0) and price 1001 (total 1)	14

The 14 tests contain no duplicates: each differs in at least one condition or at the boundary. They maximise detection because all four negative-total rules and the exact boundary are included, and they minimise redundancy (a 12.5% reduction against 16, versus thousands of random price/promo values). **Interpretation:** decision table testing exposes combinations that isolated tests cannot.

Question 9: Legacy Database Migration – Infinite Loops

Assumed logic: for each table, for each row: validate; while the row is invalid keep calling repair() (no retry limit); then migrate the row.

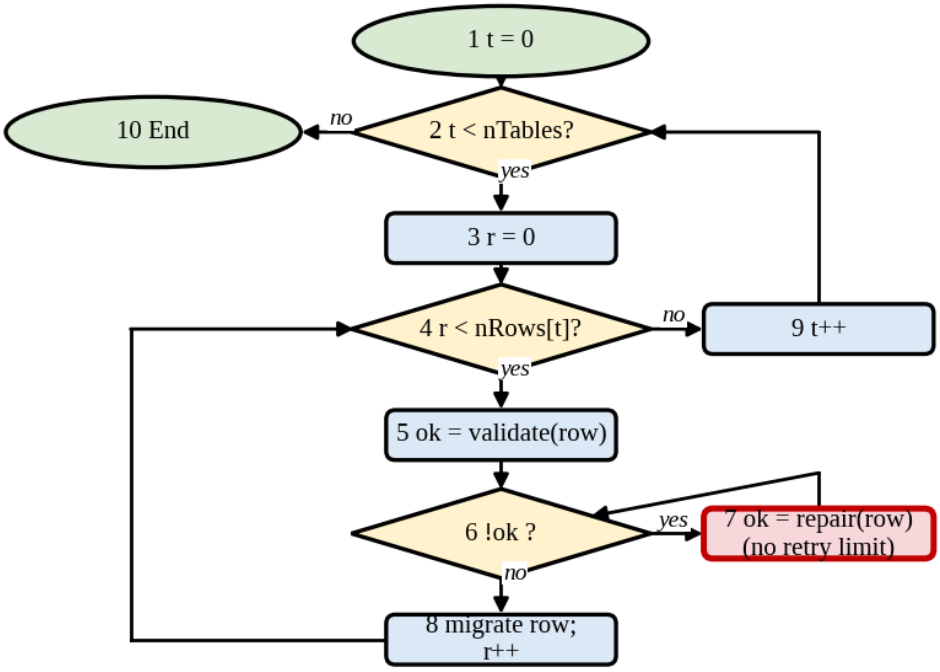


Figure 4: CFG of the migration tool (red = unbounded repair loop)

(a) Why statement coverage was insufficient

One test (1 table, 1 row that is repaired successfully on the second attempt) follows the path 1-2-3-4-5-6-7-6-8-4-9-2-10, which touches all 10 nodes and all 12 edges, so **statement coverage and branch coverage are both 100% with one test**. It never shows what happens when repair() *keeps failing* (the loop 6→7→6 never ends) or when the loops run zero or many times.

(b) Cyclomatic complexity and independent paths

E = 12, N = 10, V(G) = 12 − 10 + 2 = 4 (3 decisions + 1). V(G) = 4 is low (McCabe suggests keeping V(G) ≤ 10), so the module is testable, but loops require extra boundary tests beyond the 4 basis paths.

Path	Node sequence	Scenario
P1	1-2-10	No tables
P2	1-2-3-4-9-2-10	Table with 0 rows
P3	1-2-3-4-5-6-8-4-9-2-10	Valid row
P4	1-2-3-4-5-6-7-6-8-4-9-2-10	Row repaired

(c) Path and loop boundary tests

Test	Loop	Input	Expected
1	Outer	0 tables	Ends immediately
2	Outer / inner	1 table, 0 rows	Ends; no row processed
3	Inner	1 table, 1 valid row	Row migrated

Test	Loop	Input	Expected
4	Inner	1 table, 2 rows	Both migrated
5	Outer	2 tables	Both processed
6	Inner (n)	1 table, 10,000 rows	All migrated, r = n at exit
7	Repair	Row fixed at 1st attempt	Migrated
8	Repair	Row fixed at 2nd attempt	Migrated
9	Repair (max)	Row fixed at the retry limit (e.g. 5th)	Migrated
10	Repair (never)	repair() always fails (stub), 5 s watchdog	Terminates with error, no hang

Justification: V(G) = 4 gives the minimum number of path tests, tests 1 to 6 cover the loop boundaries (0, 1, 2, n), and tests 7 to 10 cover the repair loop, including the non-terminating case that causes the production crashes; test 10 is the one that fails on the current code and drives the fix (a retry limit). **Interpretation:** statement coverage cannot detect infinite loops; loop-boundary and path tests can.

Question 10: Food Delivery Tracker – State Transition Testing

(a) Evaluation of the current black-box strategy

ECP and BVA test input *values*. The tracker's behaviour depends on the *sequence of events*, so values cannot show what happens when a driver cancels in the "Out for Delivery" state. The state model has no defined transition for that event, so the app waits forever.

(b) Redesigned state model

Two states were added: **Cancelled** and **Reassigning Driver**, plus the events driver cancels, new driver assigned and timeout.

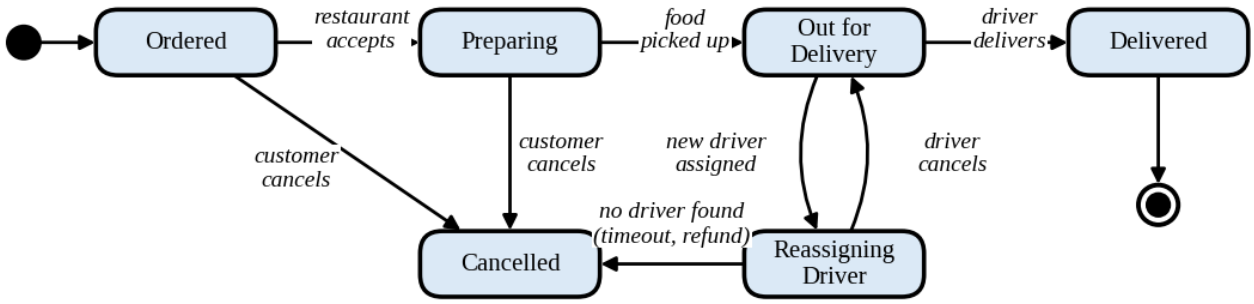


Figure 5: State-transition diagram of the real-time tracker

From	Event	To
Ordered	restaurant accepts	Preparing
Ordered	customer cancels	Cancelled
Preparing	food picked up	Out for Delivery
Preparing	customer cancels	Cancelled
Out for Delivery	driver delivers	Delivered
Out for Delivery	driver cancels	Reassigning Driver
Reassigning Driver	new driver assigned	Out for Delivery
Reassigning Driver	no driver found (timeout)	Cancelled (refund)

(c) Test scenarios

Type	Test	Expected
Valid (0-switch)	All 8 transitions above, one test each	Correct next state
Sequence (1-switch)	Out for Delivery, driver cancels, new driver assigned, delivered	Returns to Out for Delivery, then Delivered
Sequence	Out for Delivery, driver cancels, no driver, timeout	Cancelled, customer refunded
Invalid	Delivered + driver cancels; Cancelled + restaurant accepts; Out for Delivery + customer cancels	Event rejected, state unchanged
Repeated event	Driver cancels twice in a row	Second event ignored

6 states × 7 events = 42 state-event cells: 8 are legal, 34 are illegal; the 8 legal ones plus a representative set of illegal ones form the suite.

(d) Why this addresses what ECP and BVA miss

- ECP and BVA treat every input independently and cannot express *order*, *history* or *current state*.
- The stuck-app defect is a **missing transition**, found only by walking the state model and asking "what if this event occurs here?" for every state.
- Illegal transitions (for example cancelling a delivered order) are systematically listed instead of being guessed.

Interpretation: for dynamic workflows, state transition testing is the technique that checks the logic between inputs, not just the inputs themselves.

Question 11: Cybersecurity Authentication – Brute Force

Assumed logic: locked account is denied; wrong password increments failCount (lock at 5); OTP checked within 60 s; risky IP triggers a step-up check; otherwise access is granted. Observed weakness: failed OTP attempts are **not** counted.

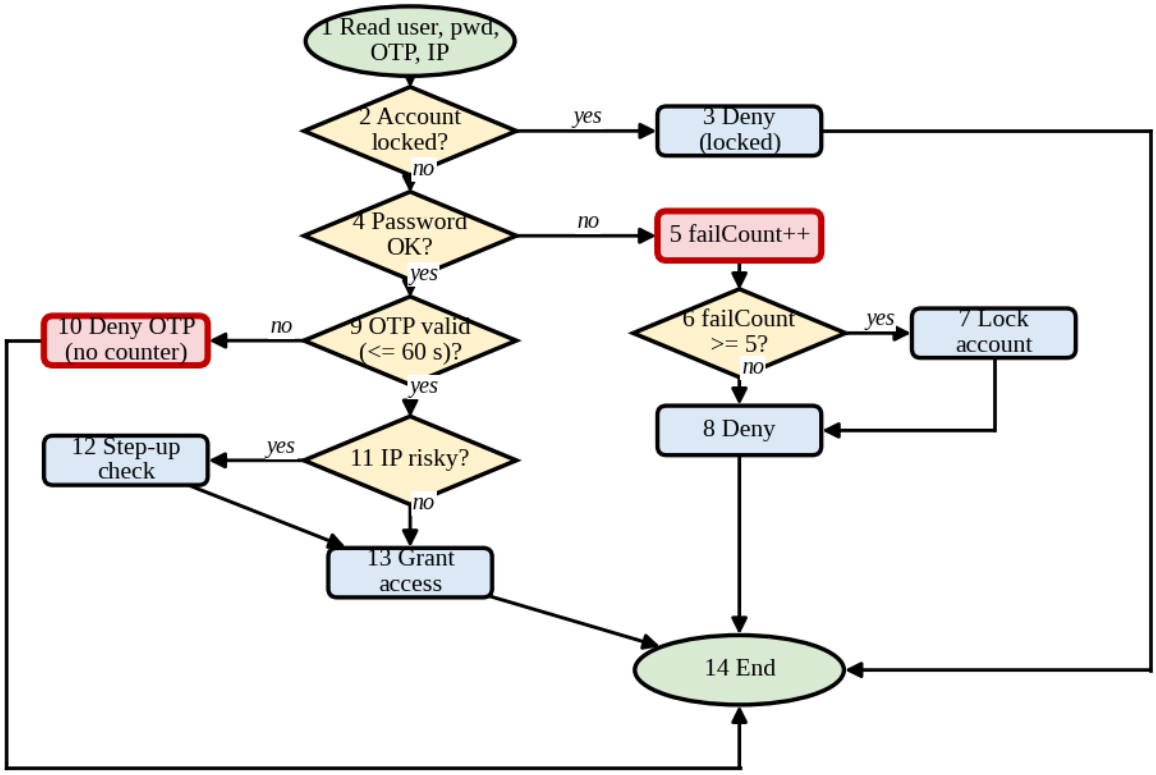


Figure 6: CFG of the authentication module (red = brute-force weak points)

(a) Assessment of the integration of methods used

Black-box tests checked valid/invalid password, OTP and IP combinations, but only a single attempt each. White-box tests covered statements, so no test repeated failed attempts; the missing counter on the OTP branch (node 10) was never noticed. Neither method modelled **repeated attempts**, which is how brute force works.

Illustrative risk: a 6-digit OTP has 1,000,000 values. At 1,000 attempts per second with no limit, an attacker tries 60,000 values in one 60-second window = 6% chance per window; over 12 windows (12 minutes) the cumulative chance is $1 - 0.94^{12} = 52.4\%$. With a shared limit of 5 attempts the chance is 5 / 1,000,000 = 0.0005% per lock-out.

(b) Formal path analysis

E = 18, N = 14, V(G) = 18 − 14 + 2 = 6 (5 decisions + 1).

Path	Node sequence	Scenario	Expected
P1	1-2-3-14	Locked account	Denied
P2	1-2-4-5-6-8-14	Wrong password, count below 5	Denied, count + 1
P3	1-2-4-5-6-7-8-14	Wrong password, 5th failure	Account locked
P4	1-2-4-9-10-14	Wrong or expired OTP	Denied; **must also count**
P5	1-2-4-9-11-12-13-14	Risky IP, step-up passed	Access granted
P6	1-2-4-9-11-13-14	Normal login	Access granted

(c) Hybrid strategy: error guessing + path analysis

Error guess (attack idea)	Test	Path targeted	Expected result
Repeat wrong OTP many times	50 wrong OTPs for one user	P4	Lock after the limit (currently fails)
Mix wrong password and wrong OTP	2 wrong passwords + 3 wrong OTPs	P2, P4	Total failures counted, lock at 5
OTP just expired / reuse	OTP at 59, 60, 61 s; reuse a used OTP	P4	59 and 60 accepted, 61 and reuse denied
Parallel attempts	100 threads guessing at once	P2, P4	Counter is atomic; lock holds
IP rotation / VPN	Same account from many IPs	P5	Limit per account, not only per IP
Injection in fields	' OR 1=1 -- in username	P1, P2	Rejected safely

Justification: error guessing supplies the attacker's scenarios, path analysis shows which code path each scenario must exercise and proves that no branch (for example node 10) is left without protection. Security posture improves because every authentication outcome has a counted, tested and limited path. **Interpretation:** brute force is a behaviour over many attempts, so single-input black-box tests and statement coverage were never going to find it.

Question 12: E-learning Tiers – Upgrade on the Day of Expiry

Assumptions: a paid subscription is active up to and including its expiry date E; Free has no expiry. The defect most likely comes from comparing with < E instead of ≤ E, so on day E the user is treated as expired.

(a) Adequacy of current criteria

Tests probably used dates well before or after expiry and never the expiry day itself, so the date partition containing the boundary was missed. Also, the combinations of current tier × new tier × date partition were not enumerated.

(b) Equivalence classes

Dimension	Classes
Date (relative to expiry E)	D1: before expiry (E − 1 or earlier) | D2: on expiry day (E) | D3: after expiry (E + 1 or later)
Current tier	Free (no expiry), Basic, Premium
Requested change	Free → Basic, Free → Premium, Basic → Premium, Basic → Basic (renew), Premium → Basic (downgrade)
Course type	Free course, Basic course, Premium course

(c) Structured test design

TC	Current tier	Date	Action	Expected
1	Free	n/a	Upgrade to Basic	Basic active, Basic courses open
2	Free	n/a	Upgrade to Premium	Premium active
3	Basic	E − 1	Upgrade to Premium	Premium immediately; unused days credited
4	Basic	**E**	Upgrade to Premium	Premium immediately; no access gap; no error
5	Basic	E + 1	Upgrade to Premium	Treated as new purchase (expired)
6	Basic	E − 1	Renew	Extends from E
7	Basic	**E**	Renew	Extends from E; no lapse
8	Basic	E + 1	Renew	Starts new period from today
9	Premium	E − 1	Downgrade to Basic	Applied at the end of the period
10	Premium	**E**	Downgrade to Basic	Applied; Premium courses locked after E
11	Premium	E + 1	Downgrade to Basic	Already expired; Basic starts

(d) Completeness

Tests 4, 7 and 10 are the boundary (day E) cases for the three paid actions. Coverage: Free tier has 2 transitions × 1 date class = 2 cases; Basic has 2 actions × 3 date classes = 6; Premium has 1 action × 3 = 3; total 11 of 11 defined cells (100%). Course access is verified after each upgrade (free, basic and premium courses) so the user sees no gap. **Interpretation:** a partition that contains only the boundary day must be tested explicitly.

Question 13: Smart-Home Lighting – Branch and Condition Coverage

Assumed logic:

```
switch (mode) {
  case AUTO: if (lux < 30 && !manual) level = 80;
    else if (lux < 70 && !manual) level = 50; else level = 0; break;
  case MANUAL: level = userLevel; break;
  case NIGHT: level = 10; break;
  default: level = 0 }
if ((level > 0 && sensorOK) || override) setDim(level); else setOff();
```

Conditions: a = lux < 30; b = !manual; c = lux < 70; e = level > 0; f = sensorOK; g = override.

(a) Risk of statement coverage only

Six tests execute every statement: AUTO with lux 20, 50 and 90; MANUAL with a sensor fault; NIGHT; an invalid mode.

Test	Mode	lux	manual	sensorOK	override	Decision results
T1	AUTO	20	F	T	F	D1 true, level 80; D3 true
T2	AUTO	50	F	T	F	D1 false, D2 true, level 50
T3	AUTO	90	F	T	F	D2 false, level 0; D3 false, setOff
T5	MANUAL	—	—	F	F	level = user; D3 false
T6	NIGHT	—	—	T	F	level 10; D3 true
T7	other	—	—	T	F	default, level 0

These 6 tests also give 100% branch coverage (switch 4 of 4; D1, D2, D3 each true and false), yet condition coverage is only **10 of 12 outcomes (83%)**: b = false (manual mode in AUTO) and g = true (override) are never tested.

(b) Condition coverage: missing logical paths

Condition	True covered by	False covered by
a lux < 30	T1	T2
b !manual	T1, T2	**missing**
c lux < 70	T2	T3
e level > 0	T1, T5	T3
f sensorOK	T1	T5
g override	**missing**	T3, T5

Add **T4**: AUTO, lux 20, manual = TRUE, override = TRUE. Result: a true, b false, so D1 and D2 are false, level 0, and D3 is true through the override: setDim(0). Now all 12 condition outcomes are covered.

(c) Improved white-box strategy

Criterion	Tests	Coverage
Statement	T1, T2, T3, T5, T6, T7	100%
Branch	same 6	100%
Condition	T1 to T7	100% (12 of 12)
Extra: boundaries	lux 29, 30, 69, 70	Both sides of each threshold

Criterion	Tests	Coverage
Extra: MC/DC for D3	e, f, g toggled one at a time	Each condition affects the outcome alone

Justification: manual mode and override are the interface between the controller, sensor hardware and user app; untested combinations of them are where hardware-software integration failures hide. Condition and boundary tests exercise every such combination (for example override with a failed sensor) before the controller is connected to real hardware. **Interpretation:** statement coverage reached 100% while 17% of the condition outcomes were untested.

Question 14: Logistics Shipping Cost – Robustness Testing

Assumptions: weight 0.1 to 50.0 kg (resolution 0.1); zones Z1 to Z4; speed Standard, Express or Overnight; cost = weight × rate (for example 40 per kg for Z1 Standard).

(a) Analysis of the failure

Validation testing covered only valid weights. With no check, a negative weight gives a negative cost: −5 kg × 40 = **−200**, which would credit the customer, and 0 kg gives a free shipment. Invalid values lie outside the valid partition, so tests that use only valid ranges never reach them.

(b) Robust boundary value analysis

Value	Position	Expected	Cost at 40/kg
−5, −0.1	Invalid (negative)	Rejected: weight must be positive	—
0.0	Below minimum	Rejected	—
0.1	Minimum	Accepted	4.00
0.2	Min+	Accepted	8.00
25.0	Nominal	Accepted	1,000.00
49.9	Max−	Accepted	1,996.00
50.0	Maximum	Accepted	2,000.00
50.1	Max+	Rejected: exceeds limit	—
abc, empty, null	Non-numeric	Rejected	—
1,000,000,000	Extreme	Rejected, no overflow	—

Other inputs: zone: Z1 to Z4 valid; 0, Z5, empty invalid; speed: three valid values; "Same-day", empty, lower-case "express" invalid. **Combinations** (one boundary at a time): 6n + 1 = 19 cases for 3 fields.

(c) Why negative scenarios are necessary

- **Financial integrity:** a negative or zero weight produces negative or zero cost; the company would pay the customer.
- **Robustness:** real users, other systems and attackers send invalid input; the system must reject it with a clear message and keep running.
- **Security:** unvalidated numeric and text input is the entry point for overflow and injection.
- **Defect economics:** invalid-input defects found after release are far more expensive than those found in test.

Interpretation: robustness testing extends BVA to the invalid side of every boundary; here it would have caught the zero and negative weight failures immediately.

Question 15: Medical Imaging – Parallel Processing and Prime Data Sizes

Assumed logic: split n items among T worker threads: chunk = n / T, rem = n % T; the first rem workers take one extra item; join and merge. The defect hides when rem ≠ 0.

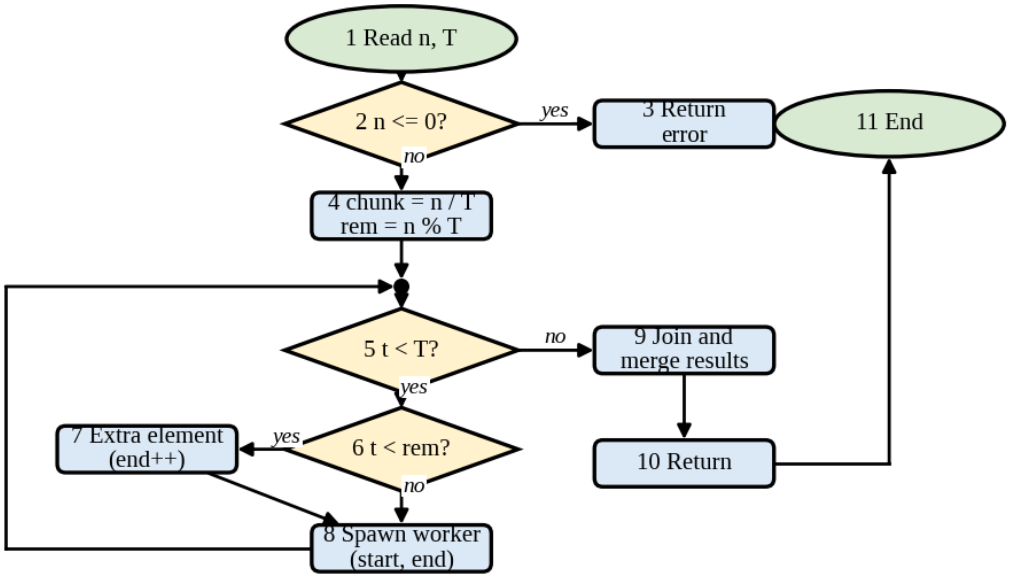


Figure 7: CFG of the parallel partitioning module

(a) Evaluation of random testing

Imaging data sizes in tests were typically 256, 512, 1024 (multiples of any power-of-two T), so rem = 0 and the remainder path (node 7) never ran. A prime n > T always has rem ≠ 0, whatever the thread count, which is why the defect appears only for prime sizes. Primes are 78,498 of the first 1,000,000 integers (7.85%), and random tests drawn from "typical" sizes almost never include them.

n (T = 4)	chunk, rem	Items per worker	Result if remainder is ignored
16	4, 0	4, 4, 4, 4	Correct
17 (prime)	4, 1	5, 4, 4, 4	16 of 17 processed: 1 item lost
13 (prime)	3, 1	4, 3, 3, 3	12 of 13 processed: 1 item lost
3 (prime)	0, 3	1, 1, 1, 0	3 of 3 needed; one worker gets an empty range
7919 (prime)	1979, 3	1980, 1980, 1980, 1979	7,916 of 7,919 processed: 3 lost

(b) Basis path testing

E = 14, N = 11 (including the join node), V(G) = 14 − 11 + 2 = 4 (3 decisions + 1).

Path	Node sequence	Scenario	Test	Expected
P1	1-2-3-11	n ≤ 0	n = 0	Error, no threads
P2	1-2-4-J-5-9-10-11	Loop not entered (T = 0)	n = 10, T = 0	Error (currently division by zero at node 4)
P3	1-2-4-J-5-6-8-J-5-9-10-11	rem = 0, no extra element	n = 16, T = 4	4, 4, 4, 4; all items processed
P4	1-2-4-J-5-6-7-8-J-5-9-10-11	rem ≠ 0, extra elements	n = 17, T = 4	5, 4, 4, 4; all 17 processed

Additional structured tests: n = 1 and n = 2 (smallest primes), n = T and n = T − 1 (boundary between empty workers and none), n = 7919 (large prime), and a prime n with T = 1 and T = 8.

(c) Mathematical accuracy and system stability

- **Accuracy:** verify that the sum of all chunk sizes equals n for every case and that the parallel result equals the serial result within a relative tolerance (for example 10^{-6}); floating-point sums depend on the order of merging, so use a fixed deterministic merge order.
- **Stability:** no deadlock, no worker blocked on an empty range, and no crash for T = 0 or n < T.
- **Why not random testing:** it cannot target the remainder path; basis path testing guarantees that all 4 independent paths, including P4, run.

Interpretation: the defect depends on a structural condition (n mod T ≠ 0), so structural (path) tests find it with certainty while random sizes find it by luck.